

RabTech Academy - Task 02

Service Blueprint, SLOs and Cloud Cost Envelope

Cloud Computing & DevOps Internship

Prepared by	Kadri Mohammad Nazar Sadikhusen
Program	BCA - Rai University
Internship	RabTech Academy - Cloud Computing & DevOps
Task	02 - Service Blueprint, SLOs & Cloud Cost Envelope

My approach

For this task, I compared the Lean, Balanced and Resilient options using the given availability, latency, user load, RPO, RTO and budget targets. I then selected the option that gives a practical balance between reliability and cost.

The scenario targets in this report come from the Task 02 SLO and cost-envelope workbook. The cost figures used below are planning assumptions for this exercise, not live cloud pricing.

1. Service Blueprint

I used a simple two-tier service model: an application layer and a database layer. A load balancer sits in front of the application instances.

Users	Request goes to the service	
Load Balancer	Receives traffic and sends it to a healthy app instance	
App 1 - AZ-1	Application instance	App 1 and App 2 share the traffic
App 2 - AZ-2	Second application instance for redundancy	
Primary DB	Main database used by the application	
Standby DB	Standby copy used for database failover	
Monitoring + Backup	Supporting services for health, alerts and recovery	

Main paths: Request: User -> Load Balancer -> App -> Database -> Response. Dependency: App -> Database. Failure: App 1 fails -> Load Balancer sends traffic to App 2. Primary DB fails -> failover to standby.

2. Scenario Targets

Scenario	Availability	P95	Users	RPO	RTO	Budget
Lean	99.0%	800 ms	500	1440 min	240 min	₹2,500
Balanced	99.5%	500 ms	5,000	60 min	60 min	₹8,000
Resilient	99.9%	300 ms	25,000	15 min	30 min	₹25,000

What these targets mean in practice: higher availability and lower latency need more infrastructure. Lower RPO/RTO also means the system needs stronger data protection and faster recovery planning.

3. Cost Envelope

Scenario	Budget	Estimated Cost	Headroom	Headroom %
Lean	₹2,500	₹2,350	₹150	6.00%
Balanced	₹8,000	₹7,500	₹500	6.25%
Resilient	₹25,000	₹23,500	₹1,500	6.00%

Headroom = Budget - Estimated Cost. Headroom % = Headroom / Budget x 100.

These estimated costs are only the assumptions used for the exercise. They should not be presented as actual AWS, Azure or GCP prices.

4. Comparison and My Decision

I did not treat one option as universally "right" or "wrong". Each one fits a different requirement. The decision depends on how much reliability and recovery capability is needed compared with the available budget.

Option	What it gives	Main limitation	My view
Lean	Lowest cost, simple single-region setup, backup	Lower availability and slower recovery	Good only when cost is the main priority
Balanced	Multi-AZ app and DB redundancy, backup/recovery, monitoring	Costs more than Lean	Best overall balance for this task
Resilient	Multi-region DR, stronger redundancy and faster recovery	Highest cost and complexity	Best for critical services with strict recovery needs

FINAL DECISION: BALANCED

I selected the Balanced architecture because it meets the middle scenario targets without taking the much higher cost of the Resilient option. It also gives a clear improvement over the single-region Lean setup by using application and database redundancy across availability zones.

5. Failure and Recovery Plan

- **Application failure:** If one application instance becomes unhealthy, the load balancer should stop sending new traffic to it and use the healthy instance.
- **Database failure:** If the primary database fails, the standby database can be promoted through the planned failover process.
- **Data recovery:** Backups are kept for recovery from deletion or corruption. Backup and replication serve different purposes.
- **Monitoring:** Health, availability, latency, errors and backup status should be monitored so that alerts can be raised when targets are at risk.

6. Conclusion

The final choice was made by looking at three things together: the SLO targets, the RPO/RTO requirements and the monthly cost envelope. Based on that comparison, Balanced is the most practical option for the given scenario.

This design keeps the architecture understandable while still showing the main ideas expected in a Cloud and DevOps service blueprint: redundancy, health checks, monitoring, backup/recovery and clear recovery objectives.